

How Validation Works in Horizon

What does the validator do?

Whenever Horizon refactors your Java code, it doesn't just trust the AI got it right. The validator runs a series of checks to make sure the refactored code is correct, safe, and actually did what you asked. Think of it as a safety net that catches mistakes before they reach you.

The Big Picture: Four Security Gates

Every refactored piece of code must pass through four gates, in order:

Gate 1 -- "Does it parse?"

First, we check if the new code is valid Java. If it can't be parsed, there's no point continuing. We try a few tricks (like wrapping bare statements in a class) because the AI sometimes outputs code fragments rather than full classes.

If it fails: We send it back to the AI with the syntax error and ask it to try again. It gets three chances. After that, we tell the Planner to come up with a different strategy.

Gate 2 -- "Did complexity get worse?"

Some refactorings should make code simpler. We use cyclomatic complexity (a standard metric that counts how many different paths through a piece of code there are -- higher numbers mean harder-to-test code).

- For most refactorings, we enforce strict: complexity must go down or stay the same.
- For some (like splitting a loop in two), we loosen the rule: complexity can go up by 1.
- For extracting methods, we use a per-method rule: the method you extracted from must get simpler.
- For inlining (merging a method back into its caller), we skip the check entirely.

If it fails: The system says 'your refactoring made the code harder to understand' and asks the AI to try a different approach.

Gate 3 -- "Did you touch code you shouldn't have?"

When you ask Horizon to refactor a specific method, the AI should only change that method. Everything else should stay exactly as it was. The validator compares the structure of every non-target method between the original and refactored code.

This gate applies the same way to every refactoring type. There are no per-refactoring rules. If a method outside the target scope changed structurally, that's a boundary violation.

If it fails: The AI is told exactly which method was accidentally modified and asked to preserve it.

Gate 4 -- "Did the refactoring actually happen?"

This is the most specific check. For each type of refactoring, the validator checks that the expected structural change occurred.

- If you asked to extract a method, there should be at least one more method in the code.
- If you asked to flatten conditionals, the maximum nesting depth should decrease.
- If you asked to rename a symbol, the structure of every method should match the original.
- And so on.

If it fails: The system tells the AI 'you said you extracted a method but the method count didn't change' and asks it to try again.

The 12 Refactoring Types and What They Mean

Refactoring	Category	What It Does
Flatten Conditional	Control Flow	Transforms deeply nested if-statements into a flat sequence of early returns or guard clauses.
Decompose Conditional	Control Flow	Breaks a complicated boolean expression into separate named boolean variables.
Consolidate Conditional	Control Flow	Merges multiple if-statements or switch cases that share identical behavior.
Remove Control Flag	Control Flow	Eliminates the 'boolean sentinel' pattern in favor of direct break or return statements.
Replace Loop with Pipeline	Control Flow	Converts an imperative for-loop into a declarative Java Stream pipeline.
Split Loop	Control Flow	Takes one loop that performs two unrelated tasks and splits it into two separate loops.
Extract Method	Method Movement	Moves a block of code into a new, descriptively named helper method.
Inline Method	Method Movement	Takes a method whose body is simple, replaces every call site with the body directly.
Extract Variable	State Management	Replaces a repeated or complex expression with a named variable computed once.
Inline Variable	State Management	Replaces a variable that just holds a simple expression result with the expression itself.
Extract Constant	State Management	Promotes a hardcoded magic number into a named static final field.
Rename Symbol	State Management	Changes the name of a variable, method, or field to be more descriptive.

Cyclomatic Complexity Rules Per Refactoring Type

Refactoring	CC Rule	What This Means
Flatten Conditional	Loosened	Complexity can increase by up to 1.
Decompose Conditional	Extract Rule	The target method's CC must not increase.
Consolidate Conditional	Strict	Overall CC must not increase.
Remove Control Flag	Strict	Overall CC must not increase.
Replace Loop with Pipeline	Strict	Overall CC must not increase.
Split Loop	Loosened	Complexity can increase by up to 1.
Extract Method	Extract Rule	The source method's CC must not increase.
Inline Method	Skip	No complexity check.
Extract Variable	Strict	Overall CC must not increase.
Inline Variable	Strict	Overall CC must not increase.
Extract Constant	Strict	Overall CC must not increase.
Rename Symbol	Strict	Overall CC must not increase.

How the CC rules are enforced

- **Strict:** If the refactored code's overall cyclomatic complexity is higher than the original, the refactoring is rejected.
- **Loosened:** Same as strict, but allows the refactored CC to exceed the original by up to 1.
- **Extract Rule:** Instead of checking overall code complexity, it checks only the specific target method.
- **Skip:** The complexity gate is bypassed entirely for this refactoring type.

Intent Check Requirements Per Refactoring Type

Refactoring	What the Validator Checks	Pass Condition	Fail Condition
Flatten Conditional	Max nesting depth of IfStatement nodes	New depth strictly less than old	Depth unchanged or increased
Decompose Conditional	BinaryOperation nodes AND new variable names	Binary ops decreased, OR at least one new variable was introduced	No new variables AND no binary-op reduction
Consolidate Conditional	IfStatement + SwitchStatement count	Count decreased	Count unchanged or increased
Remove Control Flag	BreakStatement + ReturnStatement AND variable changes	Exit points increased, OR variables removed, OR new vars added	No change in exits, no vars removed, no new vars

Replace Loop with Pipeline	Loop nodes AND Stream API calls	Loop count decreased AND stream evidence -- or loop count decreased alone	Loop count unchanged
Split Loop	ForStatement + WhileStatement count	Count increased	Count unchanged or decreased
Extract Method	MethodDeclaration count	Count increased	Count unchanged or decreased
Inline Method	MethodDeclaration count	Count decreased or stayed the same	Count increased
Extract Variable	VariableDeclarator count	Count increased	Count unchanged or decreased
Inline Variable	VariableDeclarator count	Count decreased or stayed the same	Count increased
Extract Constant	FieldDeclaration count AND uppercase names	Field count increased OR new uppercase variables	Neither increased nor new uppercase
Rename Symbol	Per-method structural signatures (SHA-256)	Every original method has matching signature in refactored code	At least one signature unmatched

Boundary Preservation

This gate has no per-refactoring rules. It works the same way for every refactoring type.

The validator extracts all methods from both the original and refactored code, then compares them side by side. Any method that exists in both versions but is not in the target scope must have an identical structural signature.

New methods are always allowed. Adding new classes, enums, or helper methods is fine. Modifying something that was supposed to stay untouched is not.

How the System Handles Failure

When a gate fails, Horizon follows a recovery strategy:

1. Syntax failure: Send the error back to the AI Generator and ask it to fix the syntax. Up to 3 attempts.
2. Structural failure (complexity, boundary, or intent): Send the finding back to the Generator once for a targeted fix.
3. Still failing after that: Escalate to the Planner to rethink the entire refactoring strategy. Up to 3 strategy retries.

If all retries are exhausted, the system reports ABORT_SYNTAX or ABORT_STRATEGY and returns the best effort so far.

Error Tiers (Severity Levels)

Failures are organized into tiers:

Tier	Name	What went wrong
Tier 1	Syntax	Generated code isn't valid Java -- can't even parse it
Tier 2-A	Complexity	Cyclomatic complexity increased beyond what's allowed
Tier 2-B	Boundary	The AI modified a method it wasn't supposed to touch
Tier 2-C	Intent	The refactoring didn't produce the expected structural change
Tier 3	Judge	The Judge model reviewed the result and rejected it

Tier 1 is the most fundamental. Tier 3 is the final human-judgment-like review performed by a separate AI model.

Real Example: What Happens During Validation

Let's say you ask Horizon to extract a method from this code:

```
class Calculator {
    void process() {
        int total = price * quantity;
        double tax = total * 0.08;
        System.out.println(total + tax);
    }
}
```

The AI might produce:

```
class Calculator {
    void process() {
        double result = calculateTotal(price, quantity);
        System.out.println(result);
    }
    double calculateTotal(int price, int quantity) {
        int total = price * quantity;
        double tax = total * 0.08;
        return total + tax;
    }
}
```

The validator then checks:

4. Syntax: Parses OK -- both are valid Java.
5. Complexity: Uses the Extract Rule -- process()'s CC must not increase. Original had 1 path, refactored has 1 path. Pass.
6. Boundary: No other methods exist to leak into. Pass.
7. Intent: Method count was 1, now it's 2. Extract-method detection passes.

All gates green -- the refactoring is accepted and moves on to the Judge for final review.